# Performance Comparison of File Access Methods in Python

INFT2201 – Introduction to Operating Systems

Mahin Hossain

16/3/2026

## Abstract

This experiment evaluated different file access techniques by searching through a dataset containing around one million used-car records. Five Python programs were implemented to compare sequential and direct access methods using both CSV and binary file formats. Memory consumption was also monitored when possible. The findings indicate that direct access on binary files performs significantly faster than sequential searches on CSV files, while all approaches maintained minimal memory usage.


## Introduction

Handling and searching large datasets is a fundamental task in many software applications. Python offers multiple ways to read and process files, and each method differs in performance and memory efficiency. CSV files store information as plain text with variable-length records, whereas binary files store data in a compact and fixed-size format.

The aim of this lab is to measure and compare the time required to locate a specific record using sequential access in a CSV file versus direct access in a binary file. By applying both techniques to the same dataset, the performance differences between them can be clearly demonstrated.

## Theoretical Background

Sequential search involves reading a file from the beginning and checking each record until the desired one is found. This approach works for both CSV and binary formats, but its performance decreases as file size increases because every preceding record must be processed.

Direct access, on the other hand, is applicable only to binary files with fixed-length records. Since each record occupies exactly 129 bytes, the position of any record can be calculated using the formula:

byte_offset = record_number × 129

The program can then jump directly to that position using seek(), allowing almost immediate retrieval regardless of file size.

Memory usage is tracked using Python's resource module, which reports peak memory consumption (RSS). Since each program processes one record at a time rather than loading the entire dataset, memory usage remains under 10 KB. It is important to note that this module is supported only on Unix-based systems and not on Windows.

## Experimental Design and Procedure

Five Python programs were executed using the *full_auto.csv* dataset, which contains approximately one million records. The experiments were conducted on Ubuntu due to compatibility with the resource module.

**findrec_csv.py**

The CSV file was opened and the header row was ignored.
The program iterated through each line while counting records until it reached record number 500,000.
Once found, the record was split into fields and displayed.
Memory usage was recorded at the end.

**findvin_csv.py**

The CSV file was opened and the header row was skipped.
Each line was checked by comparing the VIN field with the target value.
The search stopped immediately once a match was found.
Memory usage was measured afterward.

**convert_bin.py**

The CSV file was processed line by line.
Each record was converted into a fixed 129-byte format using predefined field sizes.
The processed data was saved into a binary file named *full_auto.bin*.
(Note: mileage data was read but not included in the binary file.)

**findrec_bin.py**

The binary file was opened in read mode.
The exact position of the desired record was calculated using the record number.
The program used seek() to jump directly to that position and read 129 bytes.
The data was then decoded and displayed.

**findvin_bin.py**

The binary file was read sequentially in chunks of 129 bytes.
The first 64 bytes of each record were decoded to obtain the VIN.
Each VIN was compared with the target until a match was identified.

## Analysis

The sequential search methods used with CSV files required more time because they involved scanning the file line by line from the beginning. For example, locating record 500,000 required processing half a million entries before reaching the target.

In contrast, the binary direct access method was significantly faster since it did not require scanning intermediate records. By calculating the correct byte position, the program accessed the desired record instantly using a single seek() operation.

Both VIN search programs relied on sequential access because the position of a specific VIN is not known beforehand. However, the binary version still performed better due to its fixed record size, which allows consistent and efficient traversal without parsing variable-length text.

Memory usage remained below 10 KB for the CSV programs, demonstrating that reading files one record at a time is highly efficient compared to loading entire datasets into memory.

## Lab Programs Summary

| **Program** | **File Type** | **Search Method / Task** | **Speed** |
|---|---|---|---|
| **findrec_csv.py** | CSV | Record number / Sequential Search | Slower |
| **findvin_csv.py** | CSV | VIN number / Sequential Search | Slower |

| Program | File Type | Search Method / Task | Speed |
|---|---|---|---|
| **convert_bin.py** | N/A | Takes CSV, gives binary (Writes as binary) | N/A |
| **findrec_bin.py** | Binary | Record number / Extract & Decode | **Fastest** |
| **findvin_bin.py** | Binary | VIN number / Extract first 64 bytes & Decode | Faster |

## Conclusions

This lab demonstrated that both file format and access technique play a crucial role in performance. Searching within a CSV file is slower due to the need to process records sequentially from the beginning.

In contrast, binary files combined with direct access provide much faster results, as the program can compute the exact location of a record and access it immediately.

For applications that frequently retrieve records by position in large datasets, using binary files with fixed-size records is a more efficient solution. Additionally, the experiment confirmed that low-level file processing keeps memory usage minimal, making it a practical approach for handling large-scale data.